// PROJECT_REPORT

SHADOWNET

// CYBERSECURITY DESKTOP APPLICATION

01 PROJECT OVERVIEW

ShadowNet is a Python-based cybersecurity desktop application combining two core security tools into a single tabbed GUI interface — a Network Scanner and a Password Manager. The Network Scanner allows users to scan any IP address or hostname for open TCP ports within a defined range, displaying live results in a console-style output panel. The Password Manager provides an encrypted vault protected by a master password, allowing users to securely store, view, copy and delete credentials. All data is encrypted using the Fernet (AES-128-CBC) standard before being saved to disk.

02 PROBLEM STATEMENT

In cybersecurity, two recurring needs are network reconnaissance and secure credential management. Security professionals regularly need to identify open ports on target systems to assess exposure. Simultaneously, managing multiple passwords securely is a challenge for individuals and organizations. Existing tools are often complex, expensive, or platform-dependent. ShadowNet aims to address both needs in a single lightweight, open-source Python application with an accessible GUI.

03 TECHNOLOGY STACK

Language
Python 3.x

GUI Framework
Tkinter + ttk

Encryption
cryptography — Fernet AES-128-CBC

Networking
socket module (TCP)

Threading
threading module

Storage
JSON (encrypted at rest)

IDE
Visual Studio Code

UI Design Reference
Google Stitch

04 IMPLEMENTATION

Network Scanner:

- Accept target IP and port range from GUI input fields.
- Launch a background thread to keep the GUI responsive during scanning.
- For each port, create a TCP socket and attempt connection using **connect_ex()**.
- If return value is **0**, the port is open — log the result to the live console output.
- A global boolean flag allows the Stop button to cancel the scan mid-way.

Password Manager:

- On first run, generate a random AES-128 key and save it to **secret.key**.
- Show a master password lock screen before granting vault access.
- Encrypt each credential field individually using Fernet before saving to **passwords.json**.
- At runtime, decrypt all entries and display them in a table with passwords masked as dots.
- Allow one-click clipboard copy and entry deletion with confirmation dialog.

05 CODE SNIPPETS

```
math.py — Network Scanner core
# Scan each port in the given range for port in range(start, end + 1): if not scanning: break s =
socket.socket(socket.AF_INET, socket.SOCK_STREAM) s.settimeout(0.5) result = s.connect_ex((ip,
port)) s.close() if result == 0: log(f"Port {port} ({service}): OPEN")
```

```
password_manager.py — Fernet Encryption
# Generate and save key on first run key = Fernet.generate_key() # Encrypt before saving to JSON
encrypted = fernet.encrypt(password.encode()).decode() # Decrypt when loading from JSON plain =
fernet.decrypt(encrypted.encode()).decode()
```

06 SAMPLE TEST CASES

INPUT	EXPECTED OUTPUT
IP: scanme.nmap.org | Ports: 1-100	**OPEN** — Port 22 (SSH), Port 80 (HTTP)
IP: 127.0.0.1 | Ports: 1-500	Scan complete. 0 open port(s) found.
Master Password: admin123	Vault unlocked. Credentials displayed in table.
Wrong Master Password	**DENIED** — Invalid credentials. Access denied.
Add: github.com / dev_user / pass@123	Entry encrypted and saved. Table refreshed.
Click ⧉ Copy on a row	Password copied to clipboard. Cell flashes ✔ Copied!

07 CHALLENGES FACED

Managing two separate GUI modules with shared theme colors and consistent dark styling across both tabs.

Preventing GUI freezing during long port scans by correctly implementing background threading with a stop flag.

Implementing placeholder text behavior in tkinter Entry fields, which has no native placeholder support.

Correctly wiring the Fernet key lifecycle so passwords remain decryptable across app sessions.

Converting the Stitch-generated HTML/CSS design accurately into tkinter widgets while maintaining visual fidelity.

08 RESULTS AND OBSERVATIONS

ShadowNet successfully implements both planned modules. The Network Scanner accurately detects open TCP ports on local and remote hosts, correctly labelling well-known services such as SSH, HTTP, FTP and MySQL. The Password Manager securely stores and retrieves credentials using AES-128-CBC encryption — all data on disk remains fully unreadable without the encryption key. The application runs entirely offline with a single external dependency, and the GUI stays fully responsive during scans thanks to background threading.